

which ensures that the valid word we find is more than one character long, just as we did before.

We're almost home; we just need to find whether any locations are contained in both `endsofbeginnings` and `beginningsofends`. If there are, that means that our invalid word is indeed a combination of two valid words without a space. We can use the `intersection()` function to find all elements that are shared by both lists:

---

```
pivot = list(set(endsofbeginnings).intersection(beginningsofends))
```

---

We use the `list(set())` syntax again; just like before, it's to make sure that our list contains only unique values, with no duplicates. We call the result `pivot`. It's possible that `pivot` will contain more than one element. This would mean that there are more than two possible combinations of valid words that could compose our invalid word. If this happens, we'll have to decide which combination is the one the original writer intended. This cannot be done with certainty. For example, consider the invalid word *choosespain*. It's possible that this invalid word is from a travel brochure for Iberia ("Choose Spain!"), but it's also possible that it's from a description of a masochist ("chooses pain"). Because of the huge quantity of words in our language and the numerous ways they can be combined, sometimes we can't be certain which is right. A more sophisticated approach would take into account context—whether other words around *choosespain* tend to be about olives and bullfighting or about whips and superfluous dentist appointments. Such an approach would be difficult to do well and impossible to do perfectly, illustrating again the difficulty of language algorithms in general. In our case, we'll take the smallest element of `pivot`, not because this is certainly the correct one, but just because we have to take one:

---

```
import numpy as np
pivot = np.min(pivot)
```

---

Finally, we can write one line that replaces our invalid word with the two valid component words plus a space:

---

```
textnew = text
textnew = textnew.replace(text[loc[0]:loc[1]], text[loc[0]:pivot]+'
'+text[pivot:loc[1]])
```

---